

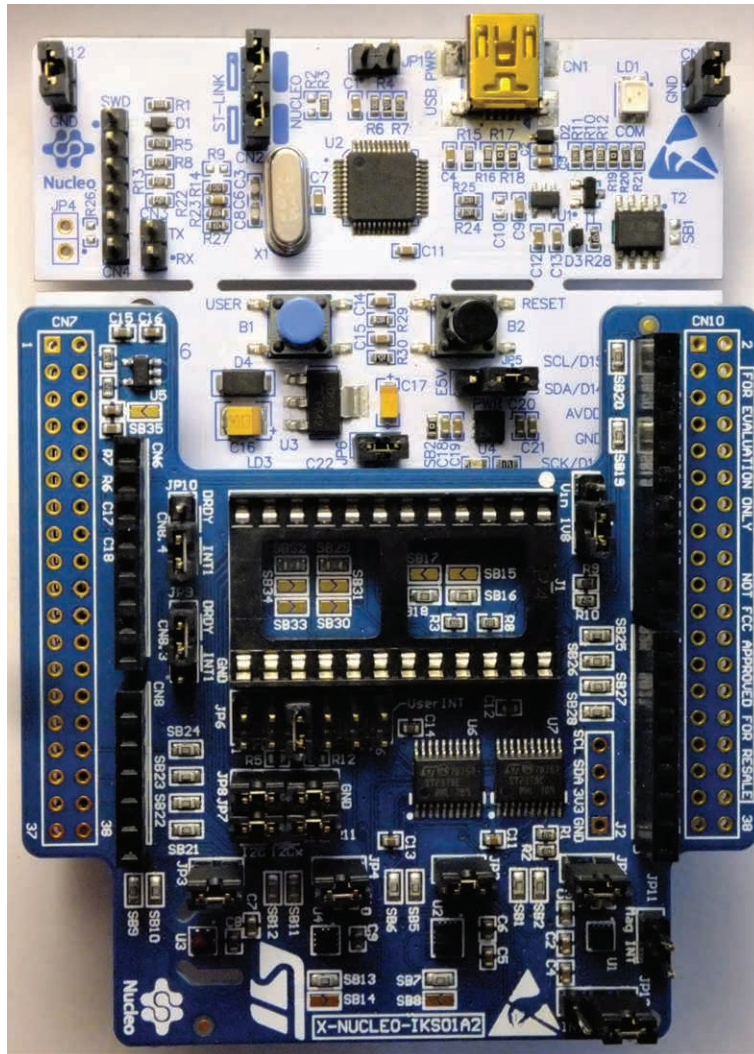


Figure 1.19 X-NUCLEO-IKS01A2 expansion board plugged into the Nucleo-F091RC development board. Photo by author.

Summary

Embedding a computer to control a system can provide benefits such as better performance and sophisticated features. We have seen three examples of applications: a hot plate for cooking food, a toy quadcopter aircraft, and a refrigerator. The embedded computer is typically made from a microcontroller that runs specialized control software which monitors the critical aspects of the system and its environment and then adjusts its outputs so the system behaves as needed. The computer

uses special hardware to interface with the inputs and outputs. To meet application requirements, embedded systems must provide sufficient concurrency, responsiveness, reliability, fault handling, and diagnostic help while meeting constraints on cost, size, weight, power, energy, and temperature. Finally, we have seen the target platform for this textbook: the Arm Cortex-M0 processor core within the STM32F091RC MCU on the STMicroelectronics' Nucleo-F091RC development board.

Exercises

1. You have been asked to add an “automatic power-off” controller to a flashlight. The flashlight should turn off automatically after 3 minutes if it is not being used any more. The controller must be small, inexpensive, and use little power.
 - a. Can you think of a way to do this with electronics but without an embedded computer?
 - b. Can you think of a way to do this with a mechanical approach?
 - c. Now, really thinking outside the box, can you think of a way to do this with a pneumatic approach (using air)?
2. Consider embedding a computer in a flashlight. List three benefits or new useful features that are now possible. How much more would you be willing to pay for each feature?
3. A modern automobile has dozens of embedded computers. However, the auto industry is extremely cost-sensitive, so there must be a compelling reason to add computers. Give five examples of features enabled by embedded computers, and explain what major benefit(s) they provide.
4. Look around and find five devices that are likely to have embedded computer control systems. Answer these questions for each device:
 - a. What does the device do?
 - b. What are the device's inputs?
 - c. What are the device's outputs?
 - d. How does the embedded computer improve the device? Does it provide more features? Does it improve performance?
 - e. What are the biggest constraints on the device (size, cost, power, etc.)?
 - f. Would it be possible to build the device using a different kind of controller (e.g. mechanical)? How would that affect the device's features, performance, cost, size, and weight?

References

- [1] STMicroelectronics NV, *User Manual UM1724: STM32 Nucleo-64 Boards*, 2019.
- [2] STMicroelectronics NV, *Reference Manual RM0091: STM32F0x1/STM32F0x2/STM32F0x8*, 2017.
- [3] STMicroelectronics NV, *STM32F091xB STM32F091xC Data Sheet*, DocID 026284, 2017.
- [4] STMicroelectronics NV, *User Manual UM2121: Getting Started with the X-NUCLEO-IKS01A2 Motion MEMS and Environmental Sensor Expansion Board for STM32 Nucleo*, DocID 029834, rev. 2, 2017.

Configuration Lock

It is possible to lock the configuration (e.g. direction, pull-up) of bits in ports A and B to prevent corruption by software bugs or glitches. After locking, only the bit's data can be changed; the configuration is fixed until the MCU is reset. Details appear in the GPIO port configuration lock register section of the reference manual [2].

Other Options

The GPIO peripheral can generate events such as interrupt and direct memory access requests based on input signals. These will be discussed in Chapters 4 and 9 and are controlled by the EXTI and SYSCFG registers.

Summary

This chapter has introduced the concepts of general purpose I/O ports and how to use them in a STM32F0 MCU using C code with the CMSIS-CORE hardware abstraction layer or the STMicroelectronics HAL hardware abstraction layer. Interfacing with inputs and outputs was introduced with examples using LEDs, a switch, and a speaker.

Exercises

1. What are the valid input voltage ranges for a STM32F0 MCU with $V_{DD} = 3\text{ V}$? With 2 V ?
2. What is the actual value of V_{DD} on your MCU board? Use a multimeter to measure this voltage.
3. Examine the schematic for your Nucleo-F091RC board:
 - a. How many GPIO port bits are available for port A?
 - b. Port B?
 - c. Port C?
 - d. Port D?
 - e. Port E?
 - f. Port F?
4. Calculate the resistor values needed to limit current through the blue and red LEDs of Figure 2.3 to 18 mA each. Assume that the supply voltage V_{DD} is 3.0 V .
5. What values need to be written to which registers in order to set a port as a digital input, with the resistive pull-up enabled?
6. Consider a program that uses bits 0 through 5 on port A as GPIO inputs, and bits 10 through 15 as GPIO outputs:
 - a. What control register settings are needed?
 - b. Write a C code to implement these control register settings.

7. Consider a system with a STM32F091RC MCU, one switch, and four LEDs. As long as the switch is not pressed, all lights shall be turned off. When the switch is pressed, the LEDs shall start to turn on one at a time (starting with LED 0), with a delay of roughly $\frac{1}{2}$ second per LED. After all the LEDs are turned on, they shall remain on until the switch is released. The selection of port bits for the switch and the LEDs is your choice. The following table shows the required sequence of LED activity, assuming the switch is pressed at time T_1 and released at time T_2 .

Time (sec)	Switch	LED0	LED1	LED2	LED3
$< T_1$	Not pressed	off	off	off	off
T_1	Pressed	on	off	off	off
$T_1 + 0.5$	Pressed	on	on	off	off
$T_1 + 1.0$	Pressed	on	on	on	off
$T_1 + 1.5$	Pressed	on	on	on	on
T_2	Not pressed	off	off	off	off

- What control register settings are needed?
- Write a C program to implement this system.

References

- [1] STMicroelectronics NV, *User Manual UM1724: STM32 Nucleo-64 Boards*, 2019.
- [2] STMicroelectronics NV, *Reference Manual RM0091: STM32F0x1/STM32F0x2/STM32F0x8*, 2017.
- [3] STMicroelectronics NV, *STM32F091xB STM32F091xC Data Sheet*, DocID 026284, 2017.

A real-time kernel (RTK) or a real-time operating system (RTOS) is designed to make it easier to create real-time systems. Preemptive scheduling is typically used to provide short response times. Prioritized task scheduling also reduces response times. The kernel is designed and built to execute with consistent and predictable timing, rather than with widely varying behavior. One example of an RTK is Keil's RTX, which is included with the Keil MDK-Arm integrated development environment.

Summary

This chapter has presented various approaches to sharing a CPU among multiple software activities. We began with a starter program with all its activities mixed together (spaghetti code). We then saw how to improve it in several important ways:

- Modularity is improved by dividing the activities into separate tasks.
- Responsiveness is improved by allowing preemption of tasks (by interrupts or other tasks), by shortening task run-times using FSMs, and by prioritizing tasks.
- CPU overhead is reduced by using hardware peripherals and by introducing a task wait state.

Exercises

1. Consider the scheduling approach described in the section "Creating and Using Tasks". See Figure 3.4. Assume there is no time taken to switch between tasks, and that the tasks have the following execution times:

Task or handler	Execution time when in flash mode	Execution time when in RGB mode
Task_Read_Switches	1 ms	1 ms
Task_Flash	100 ms	1 ms
Task_RGB	1 ms	1000 ms

- a. Describe the sequence of events that leads to maximum delay between pressing the switch and seeing the LED flash. Calculate the value of that delay.
 - b. Describe the sequence of events that leads to maximum delay between pressing the switch and seeing the LED sequence through RGB colors. Calculate the value of that delay.
 - c. What is the minimum amount of time the switch must be pressed to change the LED flashing pattern?
 - d. Would changing the scheduler to call **Task_RGB** before **Task_Flash** changes the delays in (a) and (b)? If so, determine the new delays and explain why they changed.
2. Another developer wants to add two more tasks (**Task_X** and **Task_Y**) to the system of the previous question. Each task can take up to 80 ms to complete.
 - a. When will there be maximum delay between pressing the switch and seeing the LED flash? Calculate the value of that delay.

- b. When will there be maximum delay between releasing the switch and seeing the LED sequence through RGB colors? Calculate the value of that delay.
 - c. What is the minimum amount of time the switch must be pressed to change the LED flashing pattern?
3. Consider the scheduling approach of the section “Interrupts and Event Triggering”. See Figure 3.9. Assume that `EXTI4_15_IRQHandler` starts executing as soon as the switch changes from pressed to released or from released to pressed. Also assume there is no time taken to switch between tasks or the handler, and that the tasks and handler have the following execution times:

Task or handler	Execution time when in flash mode	Execution time when in RGB mode
<code>EXTI4_15_IRQHandler</code>	0.01 ms	0.01 ms
<code>Task_Flash</code>	100 ms	1 ms
<code>Task_RGB</code>	1 ms	1000 ms

- a. Describe the sequence of events that leads to maximum delay between pressing the switch and seeing the LED flash. Calculate the value of that delay.
 - b. Describe the sequence of events that leads to maximum delay between pressing the switch and seeing the LED sequence through RGB colors. Calculate the value of that delay.
 - c. What is the minimum amount of time the switch must be pressed to change the LED flashing pattern?
4. Consider the scheduling approach of the section “Reducing Task Completion Times with Finite State Machines”. See Figure 3.11. Assume there is no time taken to switch between tasks, and that the tasks have the following execution times:

Task	Execution time when in flash mode	Execution time when in RGB mode
<code>Task_Read_Switches</code>	1 ms	1 ms
<code>Task_Flash</code>	34 ms	1 ms
<code>Task_RGB</code>	1 ms	334 ms

- a. Describe the sequence of events that leads to maximum delay between pressing the switch and seeing the LED flash. Calculate the value of that delay.
 - b. Describe the sequence of events that leads to maximum delay between pressing the switch and seeing the LED sequence through RGB colors. Calculate the value of that delay.
 - c. What is the minimum amount of time the switch must be pressed to change the LED flashing pattern?

We solve this problem by disabling preemption during the critical section. For example, we disable interrupts during the critical section in `my_good_function` as shown in Listing 4.12. As described in the section “Safely Masking Interrupts” above, we need to save the interrupt masking state, mask interrupts, and then restore the previous interrupt masking state. Preemptive kernels offer this and related methods to prevent preemption during critical sections of code.

Note that variables longer than one word will take more than one load and one store to update,¹ so they are vulnerable to an additional type of corruption: a partial update. Consider our example LED flasher system. Let’s consider changing the size of `g_RGB_delay` from one word (`uint32_t`, which is 32 bits) to two words (`uint64_t`, which is 64 bits) to allow longer time delays. The code will now need to perform two load operations to read `g_RGB_delay` from memory: `LDR R0, [g_RGB_delay]` for the low word and `LDR R1, [g_RGB_delay+4]` for the high word. Similarly, two store operations are needed to write the two words back.

If switch 2 changes while the first load instruction is executing, the data for the task will be corrupted. The CPU will complete that first load (low word) and then execute the ISR. The ISR will update the two words of `g_RGB_delay` in memory. The task code will then resume by loading R1 with the second (high) word from memory. The registers now hold corrupted data: R0 holds the old low word, and R1 holds the new high word.

Summary

In this chapter, we explored the basic organization and programmer’s model of a Cortex-M0 CPU core. We learned about general-purpose registers and special control registers in the core. We saw how memory is addressed and how a stack works. We examined the available instructions, including data movement, data processing, memory access, control flow, and miscellaneous instructions. We learned about different processor operating modes and how Thumb instructions differ from standard Arm instructions.

We then examined exceptions and interrupts. We saw the steps the CPU follows to handle exceptions: looking up a handler’s address in the vector table, entering the exception handler and then exiting it. We then covered the path that interrupts follow: from generation by a peripheral (e.g. GPIO and SYSCFG), through EXTI and the NVIC, and finally through the CPU masking hardware. Finally we discussed how to design software for interrupts: how to partition work between the ISR and main-line code, how to write software which configures the hardware to generate and recognize interrupts, and how to write the ISR. We also discussed how to handle volatile data and provide atomic object access.

Exercises

1. How does the word `0xDECD EDD1` appear in memory in a little-endian memory system as well as in a big-endian memory system? Specify the relative address for each byte.
2. Does the stack in Arm processors grow toward larger or smaller addresses?

¹ Although the Cortex-M0 supports load multiple (LDM) and store multiple (STM) instructions to process multiple registers, these instructions can be interrupted before completion.

3. Assuming that SP is 0x0000 2220 initially, what is its value after executing the instruction `PUSH {R0, R2}`?
4. Assuming that SP is 0x0000 2010 initially, what is its value after executing the instruction `POP {R0-R7, PC}`?
5. Write the Thumb code to add number five to the contents of register R6.
6. Write the Thumb code to subtract 1,000 from the contents of register R6, using R3 as a temporary register.
7. Write the Thumb code to multiply the two 32-bit values in memory at addresses 0x1234 5678 and 0x7894 5612, storing the result in address 0x2000 0010.
8. Write the Thumb code to load register R0 with the ASCII code for the letter “E” if the number in R12 is even, or “O” if it is odd.
9. Which modules generate the IRQ0, IRQ10, and IRQ30 interrupt requests, and what are their CMSIS typedef enumeration labels? Examine the interrupt vector assignments (IVA) table in the MCU reference manual [6] and the `stm32f091xc.h` file (or appropriate `device.h` file for a different MCU device).
10. We would like to configure a STM32F091RC MCU so that if interrupts IRQ0, IRQ10, and IRQ30 are requested simultaneously, the CPU responds by servicing IRQ10 first, then IRQ0, and finally IRQ30. Write the C code using CMSIS functions to configure the MCU.
11. We wish to enable IRQ13 but disable IRQ24. What values need to be loaded into which register bits, and what is the sequence of CMSIS calls to accomplish the same?
12. We wish to determine if IRQ7 has been requested. Which register and which bit will indicate this? What is the CMSIS call that will reveal the information?
13. Which register can an exception handler use to determine if it is servicing exception number 0x21? What value will the register have? What is the CMSIS interface code to read the IPSR?
14. Using EXTI_PR register we could clear the pending flags for all bits. Under what circumstances might this lead to incorrect system operation?

References

- [1] Arm Ltd., *ARMv6-M Architecture Reference Manual*, DDI 0419D, 2017.
- [2] Arm Ltd., *Cortex-M0 Devices Generic User Guide*, DUI0497A, r0p0 ed., 2009.
- [3] Arm Ltd., *Cortex-M0 Technical Reference Manual*, DDI0432C, r0p0 ed., 2009.
- [4] J. Yiu, *The Definitive Guide to the ARM Cortex-M0 and Cortex-M0+ Processors*, 2nd ed., Oxford: Newnes, 2015.
- [5] STMicroelectronics NV, *Programming Manual PM0215: STM32F0xxx Cortex-M0*, 2012.
- [6] STMicroelectronics NV, *Reference Manual RM0091: STM32F0x1/STM32F0x2/STM32F0x8*, 2017.
- [7] P. Koopman, *Better Embedded System Software*, Pittsburgh: Drumnadrochit Education LLC, 2010.
- [8] J. Ganssle, *The Art of Designing Embedded Systems*, Second ed., Elsevier Inc., 2008.
- [9] J. K. Peckol, *Embedded Systems: A Contemporary Design Tool*, John Wiley & Sons, Inc., 2008.

5